Hello Java!

Let's Code Together

by Aditya Varma

Preface

Index

- Introduction
- Java Basics
- Operators in Java
- Control Flow
- Conditionals in Java
- Loops in Java
- Arrays in Java
- Strings in Java
- Methods in Java
- Introduction to OOP in Java
- Packages in Java
- Exception Handling in Java
- File Handling in Java
- Projects

Introduction

//Introduction to Java

What is Java?

Java is a high-level, general-purpose programming language used to build everything from small applications to massive enterprise systems. You can think of Java as a universal translator, you write code once, and it automatically adapts to different devices and operating systems.

Java belongs to the category of compiled languages.
This means:

- You write code in Java.
- A compiler converts it into a special format called bytecode.
- This bytecode is then executed by the Java Virtual Machine (JVM) on any device.

You can imagine Java's compiler as a chef who prepares a basic dish (bytecode), and the JVM is like local cooks in each country adding their own flair depending on their kitchen (OS/platform).
The recipe stays the same, only the cooking varies.

Why is bytecode important?

Because bytecode is not tied to any specific device. It's a "universal" format that works everywhere, that's where Java gets its magic.

Features of Java!

1. Platform Independence

Java's famous idea is "Write Once, Run Anywhere" (WORA).

This means:

- You write Java code once.
- It gets converted into bytecode.
- That bytecode runs on any system where a JVM exists — Windows, Linux, macOS, Android, etc.

Think of bytecode like a movie subtitle file, the same subtitle file works on many different media players. Each player interprets it in its own way, but you don't need to create separate subtitles for each one. This reduces work for programmers, no need to maintain different code versions for each platform.

2. OOP Language (Object-Oriented Programming)

Java uses the object-oriented approach, which means it organizes programs into "objects", mini units that combine data and functions.

You can imagine objects like real-world things:

- A Car object has data (color, model) and actions (start, brake).
- A Student object has data (name, age) and actions (study, takeExam).

OOP helps keep programs clean, organized, and easier to expand.

3. Secure & Robust

Java takes security seriously.

It avoids many common programming hazards because:

- Memory management is automatic
- Many risky operations are restricted
- Programs run inside the JVM "sandbox," giving an extra layer of protection

Java also does automatic memory handling:

- When you create an object, Java automatically allocates memory for it.
- When the object is no longer used, the Garbage Collector recycles that memory.

You can think of it like a house where a cleaning robot (GC) quietly removes stuff you no longer need, preventing a messy room (memory leak).

This makes Java stable (robust) and safe.

4. Portable

Java programs can move from one computer to another without modification.

Since the bytecode stays the same everywhere, Java apps don't care whether they're running on:

- A laptop
- A server
- A mobile device
- A smart TV

It's as portable as carrying a USB drive with universal files plug in anywhere, and it works.

What Java is Used For?

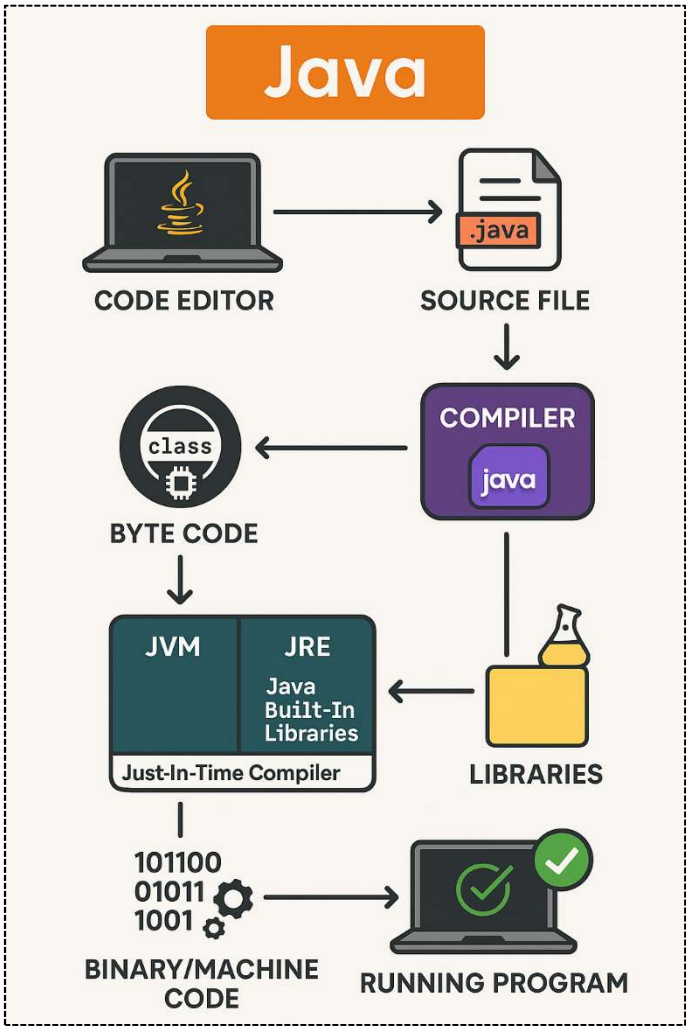
Java has been around for decades and still powers a huge part of the software world. It's used in:

- 💼 Enterprise software (banking systems, ERPs, corporate tools)
- 📱 Android apps (Java was the primary Android language for years)
- 🌐 Web applications (back-end servers using Java)
- ☁️ Cloud-based systems
- 🎮 Games (lightweight games and certain engines)
- 🖥️ Desktop applications

- 🔌 Embedded systems (smart cards, sensors, appliances)

Basically, Java is like a Swiss Army Knife — you'll find it almost everywhere in the tech world.

Write Once, Run Anywhere



//JVM, JRE & JDK – The Three Pillars of Java

When learning Java, these three names show up everywhere. They sound technical, but once you see them as parts of a mini Java ecosystem, everything becomes easy to understand.

Let's break them down with a simple analogy:

1. JVM – Java Virtual Machine

Think of the JVM as a translator who understands bytecode.

- You give JVM the bytecode produced by the compiler.
- JVM reads it, understands it, and turns it into instructions your computer can execute.
- Each operating system has its own version of the JVM, but they all understand the same bytecode.

Simple Analogy:

Imagine you write a letter in a universal language. The JVM is like a local interpreter who reads the letter and speaks in the local language (Windows, Linux, Mac). That's how Java achieves Write Once, Run Anywhere.

2. JRE – Java Runtime Environment

The JRE is the "playground" where Java programs actually run.

It includes:

- The JVM
- Built-in Java libraries (pre-written code you can use)
- Tools needed to run programs

But note: You cannot write or compile Java code with the JRE. It's only for running applications.

Analogy:

If JVM is the performer, then JRE is the stage + lights + tools that allow the performer to do their job.

3. JDK – Java Development Kit

The JDK is the complete toolset for people who want to write Java programs.

It contains:

- Everything inside the JRE
- Plus, tools for developers
    - Java compiler (javac)
    - Debuggers
    - Documentation tools

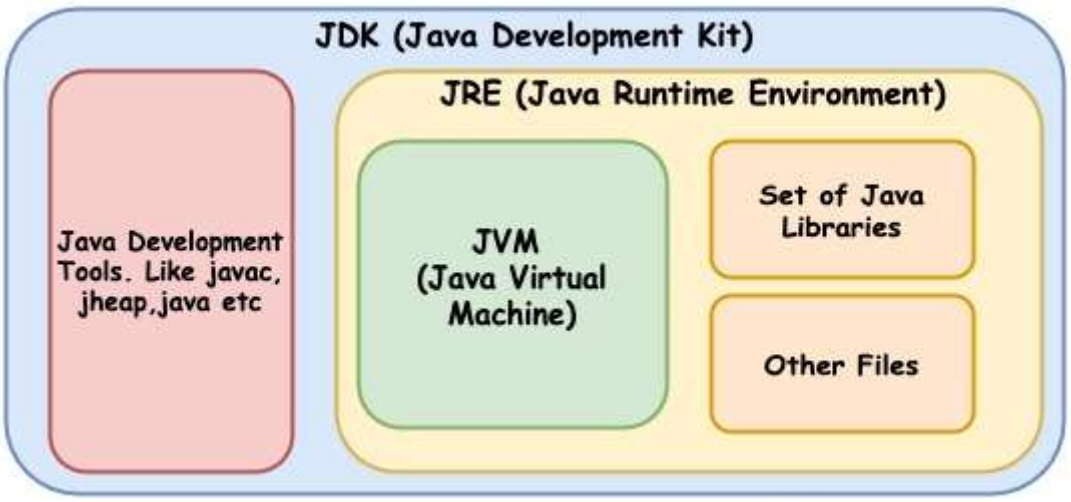
Analogy:

If JRE is the stage, then JDK is the entire theatre building, including:

- Stage (JRE)
- Backstage tools

- Equipment
- Workshops

JDK = Everything you need to create, compile, and run Java code.



//Installing Java (JDK)

Check if Java is already installed:

On Windows:

1. Open Command Prompt (CMD)
2. Type:

```
java -version
```

If Java is installed, you will see something like:

```
java version "17.0.10"
```

If Java is NOT installed, you will see:

```
'java' is not recognized as an internal or external command
```

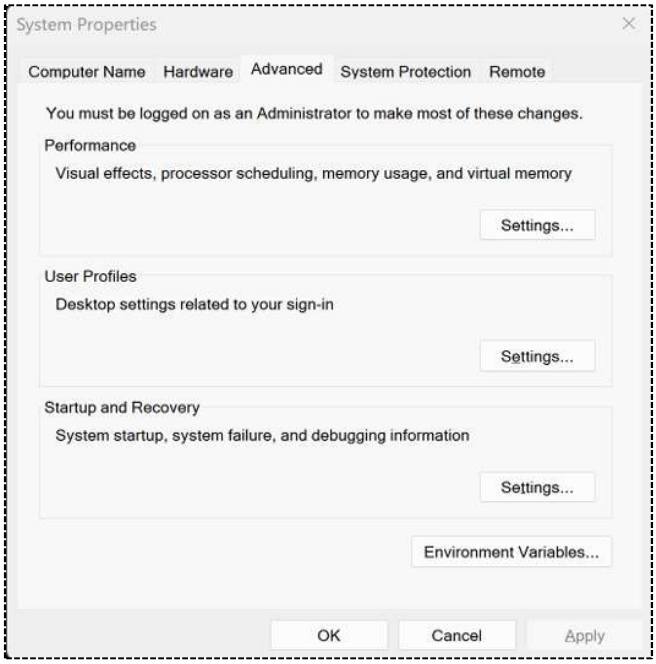
Installing Java (JDK):

1. Visit the Oracle Java SE Downloads page at https://www.oracle.com/java/technologies/downloads/.
2. Select the appropriate installer for your Windows operating system (for example, Windows x64 Installer).
3. Download the installer by clicking on the file link.
4. Run the downloaded installer (the .exe file) and follow the on-screen instructions to complete the installation.

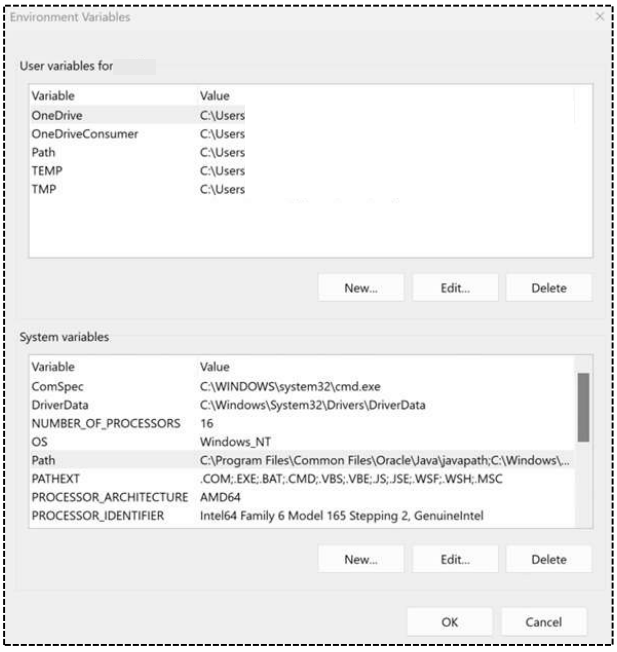
Setting Up Environment Variables:

(This step is sometimes done automatically by newer installers, but you should know it.)

1. Find the JDK installation path:
   Example path:
   `C:\Program Files\Java\jdk-25`
2. Add JAVA_HOME:
   - Search “Edit the system environment variables” -> Click Environment Variables
   - Under System variables, click New
   - Variable name: JAVA_HOME
   - Variable value: (your JDK folder path): C:\Program Files\Java\jdk-25



3. Add JDK to PATH:
    - Go to System variables → Path → Edit
    - Click New, add: `%JAVA_HOME%\bin`



4. Verify installation:
    - Open CMD and type:

```
java -version
```

You should see version numbers.

//Installing an IDE (VS Code / IntelliJ)

Option A: VS Code

1. Download and install VS Code
2. Open VS Code → Go to Extensions
3. Install:
    - Extension Pack for Java (important)
    - Debugger for Java
    - Java Test Runner (optional)

That's it VS Code is ready for Java development.

Option B: IntelliJ IDEA

(More beginner-friendly than VS Code. Handles everything automatically.)

1. Download IntelliJ IDEA Community Edition (free)
2. Install and open it
3. IntelliJ automatically detects your JDK
    - If it asks: just select the JDK folder (example: C:\Program Files\Java\jdk-17)
4. Create a New Java Project and start coding.

//Creating & Running Your First Java Program

Using CMD (Command Prompt):

Step 1: Create a new file named Hello.java in notepad.

Java requires the filename and class name to be the same.

Step 2: Write this code in the notepad,

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Step 3: Save the file in a folder you want.

Step 4: Open CMD and go to that folder.

```
cd C:\JavaPrograms
```

Step 5: Compile the file

```
javac Hello.java
```

This will create:

Hello.class (Your bytecode file)

Step 6: Run the program

```
java Hello
```

Output:

```
Hello World
```

Using IntelliJ IDEA (recommended):

Step 1: Open IntelliJ → Create New Project

- Select New Project
- Choose Java
- Select JDK 17
- Click Create

Step 2: Create a Java File

Inside src folder:

1. Right-click src
2. Select New → Java Class
3. Name it:

Step 3: Write the Program

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Step 4: Run the Program

At the top-left of the editor you will see a green play button ▶. You will get the Output.

//Basic Structure of a Java Program

Every Java program follows a specific structure. Once you understand this structure, Java becomes much easier to read and write.
Let's break it down part by part.

Understanding Class Structure:

Every Java file contains **at least one class**.

General Structure:

```
public class ClassName {
    // variables (optional)
    // methods (optional)
}
```

Points to remember:

- The class name must match the file name
  e.g., file: Hello.java → class: Hello
- A class is like a container that holds your code.
- Everything in Java lives inside a class.

Example: Think of a class as a **house**, and all your code sits inside it.

The main() Method Explained:

This is the starting point of **every** Java program.

```
public static void main(String[] args) {
    // program starts here
}
```

What each word means:

- public → Anyone can run this method
- static → Runs without creating an object
- void → Does not return anything
- main → Special method where program starts
- String[] args → Accepts inputs from the user (command-line arguments)

Think of main() as:

The **front door** of your program. Java starts running your code by entering through this door.

Compiling & Running a Java Program:

Compilation: Java code (Hello.java) → compiled into bytecode → Hello.class

javac Hello.java

Execution: Run the bytecode using the JVM

java Hello

Common Errors Beginners Face:

1. File name does not match class name:

   public class Hello {}

   but file is named: HelloWorld.java

   Fix: Rename file to Hello.java

2. Missing semicolon
3. Wrong capitalization:

   Java is **case-sensitive**.

   System ≠ system
   main ≠ Main
   String ≠ string

   Fix: Use exact uppercase/lowercase.
4. Missing curly braces { }:

   Every opening brace { must have a closing brace }.
5. Running the program with .java extension:

   You never include the .java when running.
6. Typing code outside the class:

   Everything must be **inside the class** and mostly inside the **main method** for beginners.

Quick Visual Summary:

```
public class Hello {        ← Class begins
    public static void main(String[] args) {   ← Main method begins
        System.out.println("Hello World");     ← Statement
    }                                          ← Main closes
}                                              ← Class closes
```

//Understanding Bytecode & Java Compilation Process

When you write a Java program, you type code in English-like syntax inside a .java file. But your computer cannot understand Java directly. Computers understand machine code (0s and 1s).
So, Java uses a two-step process that makes it special:

1. Java Compilation: From .java → .class
Whenever you compile a Java file using:

```
javac Program.java
```

the Java compiler (JAVAC) converts your source code into bytecode, stored in a .class file.

What is Bytecode?

- Bytecode is a universal, platform-independent code.
- It is not machine code, and not Java code — it is something in between.
- Every Java program is first converted to bytecode.

Why does Java use bytecode?

Because bytecode can run on any operating system — Windows, macOS, Linux — as long as the system has a JVM. This is why Java is called:

"Write Once, Run Anywhere (WORA)"

2. JVM Execution: From Bytecode → Machine Code
Once bytecode is created, the Java Virtual Machine (JVM) takes over.
When you run:

```
java Program
```

JVM loads the .class file and performs two main tasks:

1. Bytecode Verification
Checks if the bytecode is:

- safe
- valid
- not trying to access memory illegally
- secure for execution

2. Bytecode Execution
JVM converts bytecode into machine code specific to your operating system and CPU.
Different systems have different JVMs:

- Windows JVM
- Linux JVM
- macOS JVM

But all understand the same bytecode, which is the magic of platform independence.

How the Full Process Looks:

Here is the entire journey of a Java program simplified:

Your Code (.java)
|
COMPILED BY
|
javac (Java Compiler)
↓
Bytecode (.class)
|
EXECUTED BY
|
JVM (Java Virtual Machine)
↓
Machine Code (CPU-readable)

Why This Process Is Amazing:

✓ Portable — runs on any device
✓ Secure — JVM checks code before running
✓ Optimized — JVM improves performance at runtime
✓ Reliable — same program behaves the same everywhere

Java Basics

// Java Tokens

When you read a sentence, your brain breaks it into small parts—words, symbols, punctuation. Java also does the same. A Java program is built using tiny building blocks called *tokens*. These are the smallest units that still have meaning in code.

Think of tokens like LEGO pieces—small on their own, but powerful when connected. Java has 4 major types of tokens:

1. Keywords
2. Identifiers
3. Literals
4. Symbols & Operators

Let's explore them one by one.

Keywords:

Keywords are special words that Java has reserved for its own use. You cannot use them as variable names or class names because Java already knows what these words mean.

Examples:

class, public, static, int, void, if, else, return

Analogy:

Think of keywords as "VIP seats" in a theatre—reserved and untouchable.

Identifiers:

Identifiers are the names you give to things in your program:

- Class names
- Variable names
- Method names
- Object names

Rules (easy to remember):

- Can use letters, digits, _, $
- Cannot start with a digit
- Cannot use spaces
- Cannot use keywords
- Case-sensitive (Name, name, and NAME are different)

Identifiers are like labels you stick on your storage boxes so you know what's inside.

Literals:

Literals are fixed values written directly in the code. They don't change.

Types of literals:

- 10 → integer literal
- 3.14 → floating literal
- 'A' → character literal
- "Hello" → string literal
- true, false → boolean literal

These are the raw values that your program works with.

Symbols & Operators:

These are characters that help Java understand code structure and perform actions.

Common symbols:

- ( ) → parentheses
- {} → braces
- [ ] → brackets
- ; → statement end
- " " → string quotes

Operators do the work:

- Arithmetic: +, -, *, /, %
- Relational: ==, !=, >, <
- Logical: &&, ||, !
- Assignment: =, +=, -=

Think of operators as the tools your program uses to get things done.

Why Tokens Matter?

Tokens are the foundation of Java syntax. If Java were a language, tokens would be its alphabet. Understanding them helps you read, write, and debug programs correctly.

Lesson Program: Tokens in Java.

Source Code:

Tokens.java

```java
// Program: Tokens in Java

public class Tokens {
    public static void main(String[] args) {

        System.out.println("\n=== TOKENS IN JAVA ===");

        System.out.println("""
Java programs are made of tokens:
1. Keywords
2. Identifiers
3. Literals
4. Symbols & Operators
""");

        System.out.println("Keyword Example: public, class, static");
        System.out.println("Identifier Example: TokensInJava, count, value");

        System.out.println("Literals:");
        System.out.println(10);
        System.out.println(3.14);
        System.out.println('A');
        System.out.println("Hello Java");
        System.out.println(true);

        System.out.println("Symbols & Operators:");
        System.out.println("Semicolon ; ends statements");
        System.out.println("Using + operator: " + ("Java" + " Tokens"));
    }
}
```

//Comments in Java

When programmers write code, they also leave notes to themselves or others—just like writing sticky notes on a book page.
In Java, these notes are called comments, and the important part is:

The computer completely ignores them. They exist only for humans.

Why do we use comments?

- To explain the logic
- To make code readable
- To give instructions or warnings
- To turn OFF a line temporarily without deleting it

Java gives us 3 types of comments:

Single-Line Comment (//)

A single-line comment starts with //. Everything after it on the same line gets ignored.
Used for:

- Quick notes
- Explaining one line
- Temporarily disabling a line

Think of it as whispering a small reminder in your code.

Multi-Line Comment (/* … */)

Starts with /* and ends with */. Anything between them is ignored.
Used for:

- Long explanations
- Notes across multiple lines
- Disabling a block of code

Think of it as wrapping text inside a "big comment box."

Documentation Comment (/ … */)**

This is a special style used for generating official Java documentation using the javadoc tool.
Inside it, special tags are used:

- @author
- @version
- @param
- @return

These are used mostly in professional Java projects. Think of them as comment blocks that turn into real documentation pages.

Lesson Program: Comments in Java.

Source Code:

Comments.java

```java
// Program: Comments in Java

public class Comments {
    public static void main(String[] args) {

        System.out.println("\n=== COMMENTS IN JAVA ===");

        System.out.println("""
Types of comments:
1. Single-line
2. Multi-line
3. Documentation comments
""");

        // Single-line comment example
        System.out.println("Single-line comment example printed.");

        /*
           This is a multi-line comment.
           Used for long explanations.
        */
        System.out.println("Multi-line comment example printed.");

        /**
         * Documentation comments help create official docs.
         */
        System.out.println("Documentation comment example printed.");
    }
}
```

//Naming Conventions in Java

Naming conventions are simply rules for naming things in your Java program. They don't affect how the code runs, but they affect how it looks, how easy it is to understand, and how professional it feels. Java programs contain names for:

- classes
- methods
- variables
- constants
- packages
- interfaces

Because Java is case-sensitive, following naming conventions becomes very important. These conventions ensure your code is: Clean, Predictable, Easy to understand, Standardized across developers

Let's explore each naming rule one by one.

1. Class Naming Convention

Classes represent objects or entities, so their names should be nouns. Use PascalCase (UpperCamelCase), First letter of every word is capital, Name must be meaningful

Examples (correct):

- StudentRecord
- OrderDetails
- PaymentGateway

2. Method Naming Convention

Methods perform actions, so they should be verbs. Use camelCase, first word lowercase, next words start with uppercase

Correct:

- calculateTotal()
- getName()
- displayMessage()

3. Variable Naming Convention

Variables hold values. Their names must be clear and descriptive. Use camelCase, start with a letter, not a digit, avoid very short names (except i, j in loops), Should be easy to understand

Correct:

- studentAge
- totalMarks
- numberOfItems

4. Constant Naming Convention

Constants represent fixed values. Use UPPERCASE, use underscores between words, declare them using final

Correct:

- MAX_SPEED
- PI_VALUE

5. Package Naming Convention

Packages represent folder structures. Use all lowercase, Use meaningful names, Companies use reverse domain naming

Correct:

- student.management
- com.school.app

6. File Naming Convention

A Java file must match the public class name.

Lesson Program: Naming Conventions in Java.

Source Code:

Comments.java

```java
// Program: Java Naming Conventions

public class NamingConvention {
    public static void main(String[] args) {

        System.out.println("\n=== JAVA NAMING CONVENTIONS ===");

        System.out.println("""
1. Class → PascalCase
2. Method → camelCase
3. Variable → camelCase
4. Constant → UPPER_CASE
5. Package → lowercase
6. Interface → PascalCase
""");

        System.out.println("Correct Class Name: StudentData");
        System.out.println("Correct Method Name: calculateMarks()");
        System.out.println("Correct Variable Name: totalPrice");
        System.out.println("Correct Constant Name: MAX_LIMIT");
        System.out.println("Correct Package Name: student.management");
    }
}
```

//Escape Sequence in Java

When we write text inside System.out.println(), we usually print normal characters like letters, digits, and symbols.
But sometimes we need to print:

- a new line
- a tab space
- a double quote inside the string
- a backslash
- specially formatted text

Normal characters cannot do this directly. So Java gives us special character combinations called escape sequences.

What are Escape Sequences?

Escape sequences are special characters written using a backslash (\). They allow Java to print characters that are otherwise difficult or impossible to write directly.
Example:

\" prints a double quote
\\ prints a backslash
\n prints a new line

Common Escape Sequences in Java

1. \n New Line
   - Moves the output to the next line.
   - Works like pressing the Enter key.
2. \t Tab Space
   - Inserts a horizontal tab (same as pressing TAB).
   - Used to align data neatly.
3. \" Double Quote
   - Allows printing " inside a string.
   - Without escape sequence, Java thinks the string ends.
4. \\ Backslash
   - Prints a single backslash.
   - Because a single backslash starts an escape sequence, we need double backslash.

Why Escape Sequences Are Useful?

- They help in:
- Pretty formatting
- Writing multi-line messages
- Printing quotes inside text
- Drawing patterns
- Making console output cleaner

Escape sequences make your output professional and readable.

Lesson Program: Escape Sequence in Java.

Source Code:

EscapeSequences.java

```java
// Program: Escape Sequences in Java

public class EscapeSequences {
    public static void main(String[] args) {

        System.out.println("Line1\nLine2");          // \n -> New line
        System.out.println("Name:\tJohn");          // \t -> Tab
        System.out.println("He said, \"Hello\"");    // \" -> Double quote
        System.out.println("Path: C:\\Files");       // \\ -> Backslash

        // Combined example
        System.out.println("Start\n\tMiddle\n\t\tEnd");
    }
}
```

// Variables and Constants

In any Java program, we need a way to store information — like a number, a name, or a calculated result. Java provides two main ways to store data:

1. Variables
2. Constants (using final)

Let us understand both:

1. What is a Variable?

A variable is like a *container* that stores a value. The value can change while the program is running.

Example:

- A variable can be like a "score" in a game, it keeps changing.
- Or "age", it updates every year.

Features of Variables:

- Can store different types of data
- Value can be changed anytime
- Must have a data type
- Requires a name (follows naming conventions)

2. Declaring a Variable:

Declaration = telling Java what type of data the variable will hold. At this stage, Java only reserves memory, no value yet.

Syntax:

```
dataType variableName;
```

Examples:

```
int age;
double price;
String city;
```

3. Initializing a Variable

Initialization = giving the **first value** to the variable.

Syntax:

```
variableName = value;
```

Examples:

```
age = 25;
price = 199.99;
city = "Mumbai";
```

You can also declare and initialize in one line:

```
int number = 100;
```

4. Using Variables

Once declared and initialized, we can use variables in:

- printing
- calculations
- expressions
- method calls

Variables make programs flexible and dynamic.

5. Constants in Java (final)

A constant is a value that never changes once assigned. We create a constant using the final keyword.

Features of Constants:

- Value cannot be changed
- Must be assigned only once
- Written in UPPERCASE (naming convention)
- Used for fixed values like PI, speed limit, max range, etc.

Syntax:

```
final dataType CONSTANT_NAME = value;
```

Examples:

```
final int MAX_USERS = 500;
final double PI = 3.14159;
```

If you try to change a constant later, Java gives an error.

6. Difference Between Variables and Constants

Feature	Variable	Constant
Value	Can change	Cannot change
Keyword	None	final
Naming	camelCase	UPPER_CASE
Usage	flexible values	fixed permanent values

7. Why variables & constants matter?

They help programs become:

- dynamic
- flexible
- readable
- maintainable

Lesson Program: Variables and Constants in Java.

Source Code:

VariablesAndConstants.java

```java
// Program: Variables and Constants in Java

public class VariablesAndConstants {
    public static void main(String[] args) {

        // variable declaration + initialization
        int age = 25;
        double price = 199.99;
        String name = "John";

        // using variables
        System.out.println("Age: " + age);
        System.out.println("Price: " + price);
        System.out.println("Name: " + name);

        // constants using final
        final int MAX_STUDENTS = 100;
        final double PI = 3.14159;

        System.out.println("Max Students: " + MAX_STUDENTS);
        System.out.println("PI Value: " + PI);

        // variable changes allowed
        age = 26;
        System.out.println("Updated Age: " + age);

        // MAX_STUDENTS = 200; // ❌ Not allowed (constant)
    }
}
```

// Data Types in Java

In Java, every variable must have a data type. A data type tells Java what kind of value a variable will store, a number, decimal, character, or text.
Java data types are divided into two main groups:

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

Let's understand both:

1. Primitive Data Types

Primitive types are the simplest and most basic data types in Java. They store values directly in memory and are very fast.
Java has 8 primitive data types:

Type	Size	Description
byte	1 byte	Very small integers (-128 to 127)
short	2 bytes	Small integers
int	4 bytes	Most used integer type
long	8 bytes	Large integers (needs L)
float	4 bytes	Decimal values (needs f)
double	8 bytes	Default decimal type
char	2 bytes	Single character ('A', '#')
boolean	1 bit	true/false

Why use primitive types?

- They are fast
- They use fixed memory
- Good for calculations, numeric logic, flags, etc.

2. Non-Primitive (Reference) Data Types

These are more complex types. They do not store values directly. Instead, they store the memory address (reference) of an object.
Common non-primitive types include:

- String
- Arrays
- Classes
- Objects
- Interfaces

Characteristics:

- Begin with uppercase (String, Integer)
- Can hold multiple values
- Can be null
- Have built-in methods
- Size is not fixed

3. Difference: Primitive vs Non-Primitive

Feature	Primitive	Non-Primitive
Stores	Actual value	Memory address (reference)
Size	Fixed	Not fixed
Starts with	lowercase	Uppercase
Methods	No	Yes
Speed	Faster	Slower

Understanding this difference is important because memory and performance depend on it.

Lesson Program: Data Types in Java.

Source Code:

DataTypes.java

```java
// Program: Data Types in Java

public class DataTypes {
    public static void main(String[] args) {
        // primitive data types
        byte b = 10;
        short s = 200;
        int num = 5000;
        long bigNum = 123456789L;
        float f = 5.75f;
        double d = 99.99;
        char ch = 'J';
        boolean flag = true;

        System.out.println("byte: " + b);
        System.out.println("short: " + s);
        System.out.println("int: " + num);
        System.out.println("long: " + bigNum);
        System.out.println("float: " + f);
        System.out.println("double: " + d);
        System.out.println("char: " + ch);
        System.out.println("boolean: " + flag);

        // non-primitive types
        String name = "John Doe";
        int[] marks = {90, 85, 88};

        System.out.println("String: " + name);
        System.out.println("String length: " + name.length());
        System.out.println("Array marks: " + marks[0] + ", " + marks[1] + ", " + marks[2]);
    }
}
```

// Primitive Data Types in Java

Primitive data types are the most basic building blocks in Java. They are called "primitive" because they store simple, direct values and not objects.

Why are primitive types important?

Because they help us store:

- Whole numbers (like age, marks)
- Decimal values (like price, salary)
- Characters (like 'A', '5')
- True/false conditions

Key Features of Primitive Types

- Stores actual value directly (not memory address)
- Very fast and memory-efficient
- Names always start with lowercase (int, char, double)
- Size is fixed (decided by JVM)
- Cannot call methods on them

The 8 Primitive Data Types:

1. byte (1 byte)
   - Smallest integer type
   - Range: –128 to 127
   - Used when we want to save memory
   - Example: byte age = 25;

2. short (2 bytes)
   - Slightly bigger than byte
   - Range: –32,768 to 32,767
   - Example: short temperature = 300;

3. int (4 bytes)
   - Most commonly used
   - Range: large enough for most tasks
   - Default integer type
   - Example: int marks = 92;

4. long (8 bytes)
   - For huge numbers
   - Must end with L
   - Example: long population = 8000000000L;

5. float (4 bytes)
   - Decimal numbers with ~7 digits precision
   - Must end with f
   - Example: float price = 55.75f;

6. double (8 bytes)
   - Most commonly used decimal type
   - High precision (~15 digits)
   - Example: double salary = 45000.99;

7. char (2 bytes)
   - Stores one character
   - Uses single quotes
   - Supports Unicode values too
   - Example:

```
char letter = 'J';
char unicodeA = 65; // Prints 'A'
```

8. boolean (1 bit)
   - Stores true or false
   - Used in conditions
   - Example: boolean isJavaFun = true;

Default values (only when used as class fields)

Type	Default
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0
char	'\u0000'
boolean	false

Lesson Program: Primitive Data Types in Java.

Source Code:

PrimitvieDataTypes.java

```java
// Program: Primitive Data Types

import java.util.Scanner;

public class PrimitiveDataTypes {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // byte
        byte b = 100;
        System.out.println("byte: " + b);
        System.out.print("Enter byte: ");
        byte userByte = sc.nextByte();
        System.out.println("You entered: " + userByte);

        // short
        short s = 30000;
        System.out.println("short: " + s);
        System.out.print("Enter short: ");
        short userShort = sc.nextShort();
        System.out.println("You entered: " + userShort);

        // int
        int i = 95;
        System.out.println("int: " + i);
        System.out.print("Enter int: ");
        int userInt = sc.nextInt();
        System.out.println("You entered: " + userInt);

        // long
        long l = 8000000000L;
        System.out.println("long: " + l);
        System.out.print("Enter long: ");
        long userLong = sc.nextLong();
        System.out.println("You entered: " + userLong);

```

```
38          // float
39          float f = 99.75f;
40          System.out.println("float: " + f);
41          System.out.print("Enter float: ");
42          float userFloat = sc.nextFloat();
43          System.out.println("You entered: " + userFloat);
44
45          // double
46          double d = 55000.456;
47          System.out.println("double: " + d);
48          System.out.print("Enter double: ");
49          double userDouble = sc.nextDouble();
50          System.out.println("You entered: " + userDouble);
51
52          // char
53          char c1 = 'J';
54          char c2 = 65;
55          System.out.println("char 1: " + c1);
56          System.out.println("char 2: " + c2);
57          System.out.print("Enter char: ");
58          char userChar = sc.next().charAt(0);
59          System.out.println("You entered: " + userChar);
60
61          // boolean
62          boolean flag = true;
63          System.out.println("boolean: " + flag);
64          System.out.print("Enter boolean: ");
65          boolean userBoolean = sc.nextBoolean();
66          System.out.println("You entered: " + userBoolean);
67
68          sc.close();
69      }
70  }
```

// Input and Output in Java

Input and Output (I/O) are fundamental in programming because programs interact with users or other systems. **Input** refers to taking data from the user, files, or other sources. **Output** refers to displaying data to the user, storing results in files, or sending data to other systems.

1. Output in Java

Java provides built-in mechanisms to display information using the System.out object.
System.out.println() : Prints the text and moves to the next line.
System.out.print() : Prints text but stays on the same line.
System.out.printf() : Used for formatted output.

- Format specifiers:
  - %d → integer
  - %f → decimal
  - %s → string
  - %c → character
- Example:

```
System.out.printf("Marks: %d", 90);
```

2. Input in Java (Scanner Class)

Java takes input from keyboard, files, or command-line arguments. Most common method: Scanner class from java.util package.
You must import it:

```
import java.util.Scanner;
```

Then create a Scanner object:

```
Scanner sc = new Scanner(System.in);
```

Common Scanner methods

- nextInt() → reads integer
- nextDouble() → reads decimal
- next() → reads one word
- nextLine() → reads full sentence
- nextBoolean() → true/false

nextLine() Issue

If you use nextInt() first, it leaves a newline in the buffer. So nextLine() may capture an empty string.
Solution:

```
sc.nextLine();  // clear buffer
```

3. Command-Line Arguments

These are values you pass while running the program from terminal:

```
java MyProgram Hello 10
```

They are stored in: `String args[]`

Difference:

- `Scanner` → input during program
- `args[]` → input before program starts

4. Common Input Errors

1. Entering text when the program expects a number.
2. Forgetting `import java.util.Scanner;`
3. `nextLine()` not working due to leftover newline.
4. Using wrong input method for wrong data type.
5. Reading a character incorrectly (correct: `next().charAt(0)`).

Lesson Program: Input Output in Java.

Source Code:

InputOutput.java

```java
// Program: Input and Output in Java

import java.util.Scanner;

public class InputOutput {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // ---------------- Output Examples ----------------
        System.out.println("Welcome to Java I/O Demo!");
        System.out.print("This is printed on the same line. "); // System.out.print stays on the same line
        System.out.println("Now we move to the next line.");   // System.out.println moves to next line
        System.out.printf("Formatted number: %.2f\n", 123.456); // printf example for formatted output

        // ---------------- Input Examples ----------------
        // Example 1: Reading a String
        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        // Example 2: Reading an integer
        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        // Example 3: Reading a double
        System.out.print("Enter your height in meters: ");
        double height = sc.nextDouble();

        sc.nextLine(); // clear buffer before reading nextLine()

        // Example 4: Reading a full sentence
        System.out.print("Enter your city: ");
        String city = sc.nextLine();

        // Example 5: Reading a boolean
        System.out.print("Are you a student? (true/false): ");
        boolean isStudent = sc.nextBoolean();

        sc.nextLine(); // clear buffer before nextLine()

        // Example 6: Reading multiple words (full line)
        System.out.print("Enter your favorite quote: ");
        String quote = sc.nextLine();

        // ---------------- Display Collected Inputs ----------------
        System.out.println("\n===== USER DETAILS =====");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.printf("Height: %.2f meters\n", height);
        System.out.println("City: " + city);
        System.out.println("Student: " + isStudent);
        System.out.println("Favorite Quote: " + quote);

        sc.close();
    }
}
```

// Java Basics Summary

1. Tokens
   - Smallest units of Java: Keywords, Identifiers, Literals, Operators, Separators.
   - Form the building blocks of a program.
2. Keywords & Identifiers
   - Keywords: Reserved words (int, class, if)
   - Identifiers: Names for classes, variables, methods. Must follow rules, case-sensitive.
3. Comments
   - Single-line: //
   - Multi-line: /* ... */
   - Documentation: /** ... */
   - Improves readability; ignored by compiler.
4. Escape Sequences
   - Special sequences starting with \:
     - \n → new line, \t → tab, \" → double quote, \\ → backslash.
5. Naming Conventions
   - Class: PascalCase → StudentData
   - Method: camelCase → calculateSalary()
   - Variable: camelCase → totalMarks
   - Constant: UPPERCASE → MAX_SPEED
   - Package: lowercase → student.management
   - Interface: PascalCase → Runnable
   - File name = public class name.
6. Variables & Constants
   - Variable: Stores changeable data → int age = 25;
   - Constant: Fixed value using final → final int MAX = 100;
7. Data Types
   - Primitive: byte, short, int, long, float, double, char, boolean
   - Non-Primitive: String, arrays, objects, classes
   - Primitive stores values; non-primitive stores references.
8. Input & Output
   - Output: System.out.print(), System.out.println(), System.out.printf()
   - Input: Scanner → nextInt(), nextDouble(), nextLine(), nextBoolean()
   - Handle buffer correctly when mixing numeric and string input.
   - Command-line args: Input before execution via String[] args.

Operators in Java

In programming, we constantly perform actions like calculation, comparison, decision-making, and updating values. To do these operations, Java uses operators.

What are Operators?

Operators are special symbols that tell Java to perform an operation on values (operands). They work on variables, constants, and expressions.

Example in real life:

- + means add two items
- > means compare two values

Just like in mathematics, Java uses symbols to perform actions in a program.

Why Operators Are Important?

Operators allow Java to:

- Perform mathematical calculations
- Make comparisons and decisions
- Join conditions (true/false)
- Modify values of variables
- Assign values efficiently
- Work at low-level (bits) when needed

Without operators, a program cannot calculate, decide, or interact logically.

Classification of Operators in Java

Type	Purpose
Arithmetic	Mathematical operations
Assignment	Store and update values
Relational (Comparison)	Compare values (true/false)
Logical	Combine conditions
Unary	Operates on one value only
Ternary	Short form of if-else
Bitwise	Works on individual bits (low-level programming)

Lesson Program: Operators in Java.

Source Code:

OperatorsInJava.java

```java
// Program: Operators in Java

public class OperatorsInJava {
    public static void main(String[] args) {

        int a = 10, b = 3;

        // arithmetic
        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));

        // assignment
        int x = 5;
        x += 3;
        x *= 2;
        System.out.println("x value: " + x);

        // relational
        System.out.println("a > b: " + (a > b));
        System.out.println("a == b: " + (a == b));

        // logical
        System.out.println("(a > b) && (a == 10): " + ((a > b) && (a == 10)));
        System.out.println("(a < b) || (b == 3): " + ((a < b) || (b == 3)));

        // unary
        int n = 5;
        System.out.println("++n: " + (++n));
        System.out.println("n--: " + (n--));
        System.out.println("n now: " + n);

        // ternary
        int age = 16;
        String result = (age >= 18) ? "Adult" : "Minor";
        System.out.println(result);

        // bitwise
        int p = 5, q = 3;
        System.out.println("p & q = " + (p & q));
        System.out.println("p | q = " + (p | q));
    }
}
```

// Arithmetic & Assignment Operators in Java

What Are These Operators?

To perform calculations and store/update values, Java provides two core operator groups:

Operator Type	Purpose
Arithmetic	Performs mathematical operations
Assignment	Stores and modifies values in variables

These two categories are used in almost every Java program that deals with numbers, expressions, and data updates.

1. Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric values.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division (gives quotient)
%	Modulus (gives remainder)

Works with: int, float, double, and even char (because characters have numeric Unicode values).
Example Idea:

If a = 20, b = 6, then:

- a % b = 2 (remainder)
- 'A' + 1 = 66 because 'A' Unicode = 65

Arithmetic on floating values works the same way (e.g., 10.5 / 2.0).

2. Assignment Operators

Assignment operators store values and can update existing ones efficiently.

Operator	Meaning
=	Assign value
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulus and assign

These modify a variable directly instead of writing longer expressions.

Example Idea:

If x = 10 then:

- x += 5 → x = 15
- x *= 2 → x = 30

Works with numbers and can also work with strings for concatenation using +=.

Lesson Program: Arithmetic and Assignment in Java.

Source Code:

ArithmeticAndAssignmentOperators.java

```java
// Program: Arithmetic & Assignment Operators in Java

public class ArithmeticAndAssignmentOperators {
    public static void main(String[] args) {

        int a = 20, b = 6;
        System.out.println(a + b);
        System.out.println(a - b);
        System.out.println(a * b);
        System.out.println(a / b);
        System.out.println(a % b);

        double x = 10.5, y = 2.0;
        System.out.println(x + y);
        System.out.println(x / y);

        char c = 'A';
        System.out.println(c + 1);

        int num = 50;
        num += 10;
        num *= 2;
        num -= 5;
        num /= 5;
        num %= 3;
        System.out.println(num);

        String msg = "Hello";
        msg += " Java";
        System.out.println(msg);
    }
}
```

// Relational & Unary Operators in Java

Why These Operators Matter?

Java programs often need to compare values, check conditions, and change values step-by-step. Relational and unary operators help handle these logical and mathematical tasks.

1. Relational (Comparison) Operators:

These operators compare two values and return a boolean result (true or false). Used in conditions, decisions, loops, validations, etc.

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal

Works with numbers, characters, and Unicode values.
Example idea:

'A' < 'B' is true because character 'A' has Unicode 65, 'B' is 66.

2. Unary Operators

Unary operators operate on a single operand.

Operator	Purpose
+	Unary plus (no change)
-	Unary minus (negates value)
++	Increment (increase by 1)
--	Decrement (decrease by 1)
!	Logical NOT (reverses boolean)

Increment & Decrement Forms

Type	Meaning
++x, --x	Prefix → changes value first
x++, x--	Postfix → uses value first, then changes

Lesson Program: Relational and Unary in Java.

Source Code:

RelatoinalAndUnaryOperators.java

```java
// Program: Relational & Unary Operators in Java

public class RelationalAndUnaryOperators {
    public static void main(String[] args) {

        int a = 10, b = 7;
        System.out.println(a > b);
        System.out.println(a == b);
        System.out.println(a != b);

        System.out.println('A' < 'B');
        System.out.println('A' == 65);

        int x = 5;
        System.out.println(+x);
        System.out.println(-x);
        System.out.println(++x);
        System.out.println(x--);
        System.out.println(x);

        boolean flag = true;
        System.out.println(!flag);
    }
}
```

// Logical & Bitwise Operators in Java

Java uses operators to make decisions and also to work at the lowest binary level.

Logical Operators (Boolean Logic)

Logical operators combine conditions and return true or false. Used in conditions, decision-making, loops.

Operator	Meaning
&&	Logical AND → true only if both conditions are true
||	Logical OR
!	Logical NOT → reverses a boolean value

Short-Circuit Behaviour:

Logical operators skip the second condition if the first already decides the result.

Example idea:

- (false && something) → second part ignored
- (true || something) → second part ignored

Saves time and prevents errors like 10/0.

Bitwise Operators (Binary Level)

Bitwise operators work with bits (0s and 1s). Used in: hardware control, encryption, network programming, performance tuning.

Operator	Meaning
&	AND
|	
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Numbers are written normally but operations happen in binary.

Shift Operators

- x << n → multiply by 2^n
- x >> n → divide by 2^n

Lesson Program: Logical and Bitwise in Java.

Source Code:

LogicalAndBitwiseOperators.java

```java
// Program: Logical & Bitwise Operators in Java

public class LogicalAndBitwiseOperators {
    public static void main(String[] args) {

        int a = 10, b = 5, c = 10;
        System.out.println((a > b) && (a == c));
        System.out.println((a < b) || (b < c));
        System.out.println(!(a == c));

        int x = 5, y = 3;
        System.out.println(x & y);
        System.out.println(x | y);
        System.out.println(x ^ y);
        System.out.println(~x);
        System.out.println(5 << 1);
        System.out.println(8 >> 1);
    }
}
```

// Ternary Operator in Java

What is the Ternary Operator?

The ternary operator is a short form of if-else used when a condition decides one value out of two. Instead of writing multiple lines, the ternary lets us choose a value in a single expression.

Syntax:

```
condition ? value_if_true : value_if_false;
```

Key Points:

- Works with boolean conditions
- Returns a value, not a statement
- Best for simple choices, not long logic
- Improves readability in short decisions

Common Uses:

- Even/Odd
- Pass/Fail
- Eligibility (Age check)
- Price/Discount logic

Nested Ternary

We can use ternary multiple times to check more than one condition:

```
condition1 ? value1 :
   condition2 ? value2 :
        value3;
```

Use nested ternary carefully—too many conditions reduce readability.

Lesson Program: Ternary Operators in Java.

Source Code:

TernaryOperators.java

```java
// Program: Ternary Operator in Java

import java.util.Scanner;

public class TernaryOperator {
    public static void main(String[] args) {

        int a = 10, b = 20;
        System.out.println((a > b) ? "a is greater" : "b is greater");

        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        System.out.println((num % 2 == 0) ? "Even" : "Odd");

        int marks = 85;
        String grade = (marks >= 90 ? "A+" :
                marks >= 75 ? "A" :
                        marks >= 60 ? "B" : "C");
        System.out.println(grade);

        int age = 16;
        System.out.println((age >= 18) ? "Eligible" : "Not Eligible");

        sc.close();
    }
}
```

// Operator Precedence & Associativity in Java

When Java sees a long expression, it must decide which operator to evaluate first.
This happens through:

1. Precedence → priority order of operators
2. Associativity → direction of evaluation when precedence is the same

Both control how Java reads an expression, just like solving a maths expression.

1. Operator Precedence

Precedence decides which operator executes first.

Example idea:

```
10 + 5 * 2
```

* has higher precedence than +, so multiplication runs first → 20. If you ever want to override the default order, use parentheses, because they have the highest precedence.

2. Common Precedence Order

From highest to lowest:

1. (), ++, --, !
2. *, /, %
3. +, -
4. Relational: <, >, <=, >=
5. Equality: ==, !=
6. Logical AND: &&
7. Logical OR: ||
8. Assignment: =, +=, -=, etc. (lowest)

Java always follows this order unless parentheses change it.

3. Associativity

Associativity decides left→right or right→left evaluation when multiple operators share the same precedence.

Left to Right

Most operators: +, -, *, /, %, relational, logical &&, ||

Example idea:

50 / 5 * 2 → (50 / 5) * 2 → 20

Right to Left

Assignment and unary operators: =, +=, -=, *=, /=, ++, --

Example idea:

a = b = 10; → b = 10, then a = 10

4. Mixed Expressions

Java resolves expressions step-by-step using precedence and associativity.

Example idea:

`++a * b > 15`

- Unary first
- Arithmetic
- Relational

5. Logical Precedence Reminder

`&&` has higher precedence than `||`.

Example:

`true || false && false`

→ `false && false` → `false`

→ `true || false` → `true`

Lesson Program: Precedence and Associativity in Java.

Source Code:

PrecedenceAndAssociativity.java

```java
// Program: Operator Precedence and Associativity in Java

public class PrecedenceAndAssociativity {
    public static void main(String[] args) {

        // * has higher precedence than +
        System.out.println(10 + 5 * 2);

        // parentheses run first
        System.out.println((10 + 5) * 2);

        // precedence in mixed expression: * → + → >
        int x = 5 + 10 * 2 > 20 ? 1 : 0;
        System.out.println(x);

        // left-to-right associativity for / and *
        System.out.println(50 / 5 * 2);

        // unary ++ first, then *
        int a = 5, b = 3;
        System.out.println(++a * b > 15);

        // && runs before ||
        System.out.println(true || false && false);

        // assignment happens last due to lowest precedence
        System.out.println(10 + 2 * 3 - 4);
    }
}
```

// Type Conversion & Type Casting in Java

Java often needs to convert one data type into another while performing calculations, assignments, or reading input. These conversions follow strict rules so that values behave predictably.

There are two major mechanisms:

1. Type Conversion (Implicit / Automatic)

Java performs this automatically when it is safe.

What is Widening Conversion?

A smaller data type is converted into a larger data type.

Examples of widening chains:

byte → short → int → long → float → double
char → int → long → float → double

This conversion is safe because the larger type can store all values of the smaller type. No explicit casting is needed.

Example:

```
int x = 50;
double y = x;     // widening (safe)
```

2. Type Casting (Explicit / Manual)

You must convert manually when converting a larger type into a smaller one.

What is Narrowing Conversion?

A larger data type is converted into a smaller data type.

Examples:

double → float → long → int → short → byte
int → char
long → int

Because narrowing may cause overflow or data loss, Java requires an explicit cast:

```
double d = 9.8;
int i = (int) d;    // fractional part removed → 9
```

Example with overflow:

```
int big = 130;
byte small = (byte) big;    // wraps due to byte range
```

3. Widening vs Narrowing

Widening (Safe)	Narrowing (Risky)
Automatic	Manual
No data loss	Possible data loss
Small → large	Large → small

4. Type Promotion in Expressions

When different data types appear in the same expression, Java promotes them before evaluating.

Promotion Rules:

1. byte, short, char → int
2. If one operand is long, result is long.
3. If one is float, result is float.
4. If one is double, result is double.

Example:

```
byte a = 10, b = 20;
int c = a + b;      // a & b promoted to int

int x = 5;
double y = 2.5;
double z = x + y;   // x promoted to double
```

5. Casting Between char and int

Because a char stores Unicode numbers internally:

```
char c = 'A';    // Unicode 65
int code = c;    // widening

int n = 66;
char d = (char) n;  // narrowing
```

6. Rules You Must Always Remember

- Widening is automatic; narrowing requires explicit cast.
- Narrowing can lead to overflow or meaningless values.
- Booleans cannot be cast to or from any numeric type.
- In expressions, smaller types get promoted automatically.

Lesson Program: Type Conversion and Type Casting in Java.

Source Code:

TypeCastingAndConversion.java

```java
// Program: Type Conversion and Type Casting in Java

public class TypeCastingAndConversion {
    public static void main(String[] args) {

        // implicit widening
        int a = 50;
        double d = a;

        // explicit narrowing
        double x = 9.7;
        int y = (int) x;

        // overflow in narrowing
        int big = 130;
        byte small = (byte) big;

        // type promotion in expressions
        byte b1 = 10, b2 = 20;
        int sum = b1 + b2;              // promoted to int

        // char and int casting
        char ch = 'A';
        int code = ch;                  // widening
        char ch2 = (char) 66;           // narrowing

        // combined promotion example
        double result = b1 + a + d;

        System.out.println(d);
        System.out.println(y);
        System.out.println(small);
        System.out.println(sum);
        System.out.println(code);
        System.out.println(ch2);
        System.out.println(result);
    }
}
```

// Java Operators Summary

1. What Are Operators?

- Symbols that perform actions on values and variables.
- Used in calculations, comparisons, logic, assignments, and decision-making.

2. Arithmetic Operators

- Perform mathematical operations.
- + add, - subtract, * multiply, / divide, % remainder.
- Work with all numeric types (int, float, double, char).

3. Assignment Operators

- Store or update values.
- Basic: =
- Compound: +=, -=, *=, /=, %=
- Right-to-left associativity, a = b = 10 assigns 10 to both.

4. Relational Operators

- Compare two values, return boolean.
- >, <, >=, <=, ==, !=
- Used in conditions and loops.

5. Logical Operators

- Combine boolean conditions.
- && AND → true only if both true
- || OR → true if at least one true
- ! NOT → reverses boolean
- Support short-circuit evaluation.

6. Unary Operators

- Operate on a single operand.
- +, - (sign)
- ++, -- (pre & post increment/decrement)
- ! (boolean NOT)

7. Ternary Operator

- Compact conditional expression.
- condition ? value1 : value2
- Returns a value, not a statement.
  Example: grade = (marks >= 40) ? "Pass" : "Fail";

8. Bitwise Operators (Intro Only)

- Work at binary (bit) level.
- & AND, | OR, ^ XOR, ~ NOT
- << left shift (multiply by 2)
- >> right shift (divide by 2)

9. Operator Precedence

- Defines order of evaluation.
- Highest → () → unary → * / % → + - → relational → equality → && → || → assignment.
- Higher precedence executes first.

10. Associativity Rules

- Left-to-right: arithmetic, relational, logical.
- Right-to-left: assignment, unary.

11. Best Practices

- Use parentheses to avoid confusion.
- Understand precedence to prevent logic errors.
- Use ternary for simple decisions only.

Control Flow in Java

Lesson Program: Control Flow in Java.

Source Code:

ControlFlowInJava.java

```java
// Program: Control Flow Statements in Java

public class ControlFlowIntroduction {
    public static void main(String[] args) {

        // decision-making using if-else
        int age = 17;
        if (age >= 18) {
            System.out.println("Eligible to vote");
        } else {
            System.out.println("Not eligible to vote");
        }

        // looping using for loop
        System.out.println("For loop execution:");
        for (int i = 1; i <= 3; i++) {
            System.out.println("Iteration: " + i);
        }

        // loop with break
        System.out.println("Loop with break:");
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                break;   // exit loop
            }
            System.out.println(i);
        }

        // loop with continue
        System.out.println("Loop with continue:");
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue; // skip iteration
            }
            System.out.println(i);
        }
    }
}
```

Conditionals in Java

Lesson Program: Conditional Statement in Java.

Source Code:

ConditionalsInJava.java

```java
// Program: Conditional Statements in Java

public class ConditionalStatements {
    public static void main(String[] args) {

        int marks = 40;

        // simple if condition
        if (marks >= 35) {
            System.out.println("Pass");
        }

        // if-else condition
        int age = 16;
        if (age >= 18) {
            System.out.println("Eligible to vote");
        } else {
            System.out.println("Not eligible to vote");
        }

        // multiple conditions idea
        int number = -5;
        if (number > 0) {
            System.out.println("Positive");
        } else {
            System.out.println("Negative or Zero");
        }
    }
}
```

Lesson Program: If Statement in Java.

Source Code:

IfStatement.java

```java
// Program: if Statement in Java

import java.util.Scanner;

public class IfStatement {
    public static void main(String[] args) {

        int number = 10;

        // simple condition
        if (number > 0) {
            System.out.println("Positive number");
        }

        Scanner sc = new Scanner(System.in);

        // user input check
        System.out.print("Enter age: ");
        int age = sc.nextInt();

        if (age >= 18) {
            System.out.println("Eligible to vote");
        }

        // logical condition
        int marks = 40;
        int attendance = 80;

        if (marks >= 35 && attendance >= 75) {
            System.out.println("Student passed");
        }

        // boolean condition
        boolean isLoggedIn = true;
        if (isLoggedIn) {
            System.out.println("Welcome user");
        }

        sc.close();
    }
}
```

Lesson Program: If Else Statement in Java.

Source Code:

IfElseStatement.java

```java
// Program: if-else Statement in Java

import java.util.Scanner;

public class IfElseStatement {
    public static void main(String[] args) {

        int number = 15;

        // even or odd check
        if (number % 2 == 0) {
            System.out.println("Even");
        } else {
            System.out.println("Odd");
        }

        Scanner sc = new Scanner(System.in);

        // pass or fail
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        if (marks >= 35) {
            System.out.println("Pass");
        } else {
            System.out.println("Fail");
        }

        // login validation
        int correctPin = 1234;
        System.out.print("Enter PIN: ");
        int pin = sc.nextInt();

        if (pin == correctPin) {
            System.out.println("Login successful");
        } else {
            System.out.println("Invalid PIN");
        }

        sc.close();
    }
}
```

Lesson Program: If Else If Ladder Statement in Java.

Source Code:

ElseIFLadderStatement.java

```java
// Program: if-else-if Ladder in Java

import java.util.Scanner;

public class ElseIfLadderStatement {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // grade calculation
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        if (marks >= 90) {
            System.out.println("Grade: A+");
        } else if (marks >= 75) {
            System.out.println("Grade: A");
        } else if (marks >= 60) {
            System.out.println("Grade: B");
        } else if (marks >= 50) {
            System.out.println("Grade: C");
        } else {
            System.out.println("Result: Fail");
        }

        // age group classification
        System.out.print("Enter age: ");
        int age = sc.nextInt();

        if (age < 13) {
            System.out.println("Child");
        } else if (age < 20) {
            System.out.println("Teenager");
        } else if (age < 60) {
            System.out.println("Adult");
        } else {
            System.out.println("Senior Citizen");
        }

        sc.close();
    }
}
```

Lesson Program: Nested Conditional Statement in Java.

Source Code:

NestedConditionals.java

```java
// Program: Nested Conditional Statements in Java

import java.util.Scanner;

public class NestedConditionals {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // login and role check
        boolean isLoggedIn = true;
        String role = "admin";

        if (isLoggedIn) {
            if (role.equalsIgnoreCase("admin")) {
                System.out.println("Admin access granted");
            } else {
                System.out.println("User access granted");
            }
        } else {
            System.out.println("Please login first");
        }

        // exam result with distinction
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        if (marks >= 35) {
            System.out.println("Result: PASS");

            if (marks >= 75) {
                System.out.println("Distinction awarded");
            }
        } else {
            System.out.println("Result: FAIL");
        }

        sc.close();
    }
}
```

Lesson Program: Switch Statement in Java.

Source Code:

SwitchStatement.java

```java
// Program: switch Statement in Java

import java.util.Scanner;

public class SwitchStatement {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // day of week
        System.out.print("Enter day number (1-7): ");
        int day = sc.nextInt();

        switch (day) {
            case 1: System.out.println("Monday"); break;
            case 2: System.out.println("Tuesday"); break;
            case 3: System.out.println("Wednesday"); break;
            case 4: System.out.println("Thursday"); break;
            case 5: System.out.println("Friday"); break;
            case 6: System.out.println("Saturday"); break;
            case 7: System.out.println("Sunday"); break;
            default: System.out.println("Invalid day");
        }

        // calculator menu
        System.out.print("\nEnter two numbers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        System.out.print("Choose operation (1:+  2:-  3:*  4:/): ");
        int choice = sc.nextInt();

        switch (choice) {
            case 1: System.out.println("Result = " + (a + b)); break;
            case 2: System.out.println("Result = " + (a - b)); break;
            case 3: System.out.println("Result = " + (a * b)); break;
            case 4: System.out.println("Result = " + (a / b)); break;
            default: System.out.println("Invalid choice");
        }

        sc.close();
    }
}
```

Lesson Program: Enhanced Switch Statement in Java.

Source Code:

EnhancedSwitchStatement.java

```java
// Program: Enhanced switch Statement in Java

import java.util.Scanner;

public class EnhancedSwitchStatement {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // month days
        System.out.print("\nEnter month number (1-12): ");
        int month = sc.nextInt();

        switch (month) {
            case 1, 3, 5, 7, 8, 10, 12 -> System.out.println("31 days");
            case 4, 6, 9, 11           -> System.out.println("30 days");
            case 2                     -> System.out.println("28 days");
            default                    -> System.out.println("Invalid month");
        }

        // calculator using yield
        System.out.print("\nEnter operation (+, -, *, /): ");
        char op = sc.next().charAt(0);

        System.out.print("Enter two numbers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        int result = switch (op) {
            case '+' -> a + b;
            case '-' -> a - b;
            case '*' -> a * b;
            case '/' -> {
                if (b == 0) yield 0;
                yield a / b;
            }
            default -> 0;
        };

        System.out.println("Result: " + result);

        sc.close();
    }
}
```

// Java Conditionals Summary

- Conditional statements allow a program to **make decisions** based on conditions.
- A **condition** is an expression that evaluates to true or false.
- Conditions are formed using:
  - Relational operators (>, <, >=, <=, ==, !=)
  - Logical operators (&&, ||, !)
- Conditional statements control the **flow of execution** in a program.

Types of Conditional Statements:

- **if** : Executes code only when the condition is true.
- **if-else** : Provides two-way decision making.
- **if-else-if ladder :** Used for multiple conditions checked in sequence.
- **Nested if :** One conditional inside another for multi-level decisions.
- **switch statement** : Used when comparing a single value against multiple fixed options.
- **Enhanced switch :** Modern, cleaner version of switch with arrow syntax and no fall-through.

Important Points:

- Only **one block executes** in a conditional chain.
- Conditions must always be **boolean** in Java.
- Order of conditions matters, especially in if-else-if ladders.
- break is required in traditional switch but **not** in enhanced switch.
- Conditional statements are essential for:
  - validation
  - decision making
  - user interaction
  - real-world logic

Loops in Java

Looping statements are used when a block of code needs to be executed repeatedly based on a condition. Instead of writing the same code multiple times, loops provide a controlled and efficient way to repeat logic.

In real-world programming, repetition is common:

- Displaying records
- Processing lists
- Retrying operations
- Running tasks until a condition is met

What is a Loop?

A loop is a control structure that repeatedly executes a block of code as long as a condition remains true. Every loop consists of:

1. Initialization – starting value
2. Condition – decides continuation
3. Update – changes loop control variable

Why Loops are Needed

Without loops:

- Code becomes repetitive
- Programs are longer and harder to maintain

With loops:

- Less code
- Better readability
- Efficient execution
- Easy scalability

Types of Looping Statements in Java

Java provides three looping statements:

- for loop – when number of iterations is known
- while loop – when iterations depend on a condition
- do-while loop – when code must execute at least once

Loop Control Statements

- break → exits the loop
- continue → skips current iteration
- return → exits method

Loops form the backbone of iterative programming.

Lesson Program: Looping Statement in Java.

Source Code:

LoopingStatement.java

```java
// Program: Looping Statements in Java

public class LoopingStatements {
    public static void main(String[] args) {

        System.out.println("Looping Statements:");

        // for loop example
        System.out.println("For Loop:");
        for (int i = 1; i <= 3; i++) {
            System.out.println("Iteration: " + i);
        }

        // while loop example
        System.out.println("While Loop:");
        int j = 1;
        while (j <= 3) {
            System.out.println("Count: " + j);
            j++;
        }

        // do-while loop example
        System.out.println("Do-While Loop:");
        int k = 1;
        do {
            System.out.println("Step: " + k);
            k++;
        } while (k <= 3);

        // break example
        System.out.println("Break Example:");
        for (int x = 1; x <= 5; x++) {
            if (x == 3) break;
            System.out.println(x);
        }

        // continue example
        System.out.println("Continue Example:");
        for (int y = 1; y <= 5; y++) {
            if (y == 3) continue;
            System.out.println(y);
        }

    }
}
```

Lesson Program: For Loop Statement in Java.

Source Code:

ForLoop.java

```java
// Program: for Loop in Java

import java.util.Scanner;

public class ForLoop {
    public static void main(String[] args) {

        // printing numbers from 1 to 5
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }

        // printing even numbers from 2 to 10
        for (int i = 2; i <= 10; i += 2) {
            System.out.println(i);
        }

        // sum of numbers from 1 to n
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += i;
        }
        System.out.println("Sum = " + sum);

        // multiplication table
        System.out.print("Enter table number: ");
        int num = sc.nextInt();

        for (int i = 1; i <= 5; i++) {
            System.out.println(num + " x " + i + " = " + (num * i));
        }

        sc.close();
    }
}
```

Lesson Program: Enhanced For Loop Statement in Java.

Source Code:

EnhancedForLoop.java

```java
// Program: Enhanced for Loop in Java

public class EnhancedForLoop {
    public static void main(String[] args) {

        // Sample array
        int[] numbers = {10, 20, 30, 40, 50};

        // Example 1: Printing elements
        System.out.println("Printing elements:");
        for (int n : numbers) {          // n gets one value at a time
            System.out.println(n);
        }

        // Example 2: Sum of elements
        int sum = 0;
        for (int n : numbers) {
            sum += n;                    // add each value
        }
        System.out.println("Sum = " + sum);

        // Example 3: Conditional check
        System.out.println("Values greater than 25:");
        for (int n : numbers) {
            if (n > 25) {
                System.out.println(n);
            }
        }
    }
}
```

Lesson Program: While Loop Statement in Java.

Source Code:

WhileLoop.java

```java
// Program: while Loop in Java

import java.util.Scanner;

public class WhileLoop {
    public static void main(String[] args) {

        // printing numbers from 1 to 5
        int i = 1;
        while (i <= 5) {
            System.out.println(i);
            i++;
        }

        // sum of numbers from 1 to n
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        int sum = 0;
        int num = 1;
        while (num <= n) {
            sum += num;
            num++;
        }
        System.out.println("Sum = " + sum);

        // password retry simulation
        sc.nextLine(); // clear buffer
        String password = "java";
        String input = "";

        while (!input.equals(password)) {
            System.out.print("Enter password: ");
            input = sc.nextLine();
        }
        System.out.println("Access granted");

        sc.close();
    }
}
```

Lesson Program: Do While Loop Statement in Java.

Source Code:

DoWhileLoop.java

```java
// Program: do-while Loop in Java

import java.util.Scanner;

public class DoWhileLoop {
    public static void main(String[] args) {

        // print numbers from 1 to 5
        int i = 1;
        do {
            System.out.println(i);
            i++;
        } while (i <= 5);

        // menu-driven loop
        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            System.out.println("\n1. Balance\n2. Deposit\n3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
        } while (choice != 3);

        // password retry
        sc.nextLine(); // clear buffer
        String pass, correct = "java";
        do {
            System.out.print("Enter password: ");
            pass = sc.nextLine();
        } while (!pass.equals(correct));

        System.out.println("Access granted");
        sc.close();
    }
}
```

Lesson Program: Nested Loops in Java.

Source Code:

NestedLoops.java

```java
// Program: Nested Loops in Java

public class NestedLoops {
    public static void main(String[] args) {

        System.out.println("\n=== NESTED LOOPS DEMO ===");

        // Example 1: 3x3 grid
        System.out.println("\n3x3 Grid:");
        for (int i = 1; i <= 3; i++) {        // outer loop (rows)
            for (int j = 1; j <= 3; j++) {    // inner loop (columns)
                System.out.print("* ");
            }
            System.out.println();
        }

        // Example 2: Multiplication tables (1 to 3)
        System.out.println("\nMultiplication Tables:");
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 5; j++) {
                System.out.println(i + " x " + j + " = " + (i * j));
            }
            System.out.println();
        }

        // Example 3: Pattern
        System.out.println("Triangle Pattern:");
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

Lesson Program: Control Statement in Java.

Source Code:

ControlStatement.java

```java
// Program: Control Statements (break & continue)

import java.util.Scanner;

public class ControlStatements {
    public static void main(String[] args) {

        System.out.println("\n=== CONTROL STATEMENTS ===");

        // continue example
        System.out.println("\nUsing continue:");
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                continue;           // skip 5
            }
            System.out.println(i);
        }

        // break in while
        Scanner sc = new Scanner(System.in);
        String password = "java";

        while (true) {
            System.out.print("\nEnter password: ");
            if (sc.nextLine().equals(password)) {
                break;              // exit loop
            }
            System.out.println("Wrong password");
        }

        System.out.println("Access granted");

        // continue in while
        System.out.println("\nOdd numbers:");
        int n = 0;
        while (n < 10) {
            n++;
            if (n % 2 == 0) continue;   // skip even
            System.out.println(n);
        }

        sc.close();
    }
}
```

// Java Loops Summary

- Loops are used to execute a block of code repeatedly
- They help reduce code repetition and improve efficiency
- Every loop has three core parts:
    - Initialization
    - Condition
    - Update

Types of Loops in Java

- for loop
    - Used when the number of iterations is known
    - Initialization, condition, and update in one line
- while loop
    - Used when iterations are not known in advance
    - Condition checked before execution (entry-controlled)
- do-while loop
    - Executes code at least once
    - Condition checked after execution (exit-controlled)

Nested Loops

- A loop inside another loop
- Inner loop runs fully for each iteration of outer loop
- Used in:
    - Tables
    - Patterns
    - Matrices

Loop Control Statements

- break
    - Exits the loop immediately
- continue
    - Skips the current iteration

Important Points

- Wrong condition or update can cause infinite loops
- Off-by-one errors are common
- Proper indentation improves readability
- Choose loop type based on problem need

Arrays in Java

In programming, we often need to store and process **multiple values of the same type**. For example, marks of students, salaries of employees, temperatures of a week, or scores in a game. Declaring separate variables for each value becomes inefficient, difficult to manage, and error-prone. To solve this problem, Java provides a powerful data structure called an **Array**.

An **array** is a collection of **fixed-size**, **same-type elements** stored in **contiguous memory locations**. Instead of creating many individual variables, an array allows us to store multiple values under a single name and access them using an **index**. This makes programs more organized, readable, and efficient.

Arrays in Java are **objects**, which means they are created dynamically at runtime and stored in **heap memory**. Each element in an array can be accessed using its index, where indexing always starts from **0**. This allows fast and direct access to any element, making arrays very efficient for storing and retrieving data.

Java supports different types of arrays, such as **single-dimensional arrays**, **multi-dimensional arrays**, and **arrays of objects**. Arrays can store **primitive data types** (int, float, char, etc.) as well as **reference types** (String, objects, etc.). However, all elements in a single array must be of the **same data type**, which ensures type safety.

The size of an array is **fixed at the time of creation** and cannot be changed later. This makes arrays suitable when the number of elements is known in advance. For situations where the size needs to change dynamically, Java provides other data structures like ArrayList, which are built on top of arrays.

Arrays play a **fundamental role** in Java programming. They are widely used in:

- looping and iteration
- searching and sorting algorithms
- data processing
- matrix operations
- handling large datasets efficiently

Understanding arrays is essential before learning advanced concepts such as **collections**, **data structures**, and **algorithms**. A strong foundation in arrays helps in writing efficient programs and in understanding how data is stored and manipulated in memory.

This chapter will explore arrays step by step, starting from basic concepts and gradually moving toward more advanced array operations.

Lesson Program: Arrays in Java.

Source Code:

ArraysInJava.java

```java
// Program: Arrays in Java

public class ArraysInJava {
    public static void main(String[] args) {

        // Declaration
        int[] numbers;
        String[] names;

        // Creation
        numbers = new int[3];

        // Initialization
        numbers[0] = 10;
        numbers[1] = 20;
        numbers[2] = 30;

        // Accessing elements
        System.out.println(numbers[0]);
        System.out.println(numbers[1]);
        System.out.println(numbers[2]);

        // Declaration + Initialization
        int[] marks = {80, 85, 90};
        System.out.println(marks[1]);

        // Array of Strings
        String[] cities = {"Mumbai", "Delhi", "Pune"};
        System.out.println(cities[0]);

        // Default values
        int[] defaults = new int[2];
        System.out.println(defaults[0]); // 0
    }
}
```

Lesson Program: Arrays Operations in Java.

Source Code:

OperationsInArrays.java

```java
// Program: Array Operations in Java

import java.util.Scanner;

public class ArrayOperations {
    public static void main(String[] args) {

        int[] marks = {78, 85, 92, 66, 88};

        // Traversal using for loop
        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }

        // Traversal using enhanced for loop
        for (int val : marks) {
            System.out.println("Value: " + val);
        }

        // Reading values from user
        Scanner sc = new Scanner(System.in);
        int[] numbers = new int[5];

        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = sc.nextInt();
        }

        // Updating element
        numbers[2] = 100;

        // Sum and Average
        int sum = 0;
        for (int v : numbers) {
            sum += v;
        }
        double avg = (double) sum / numbers.length;

        System.out.println("Sum = " + sum);
        System.out.println("Average = " + avg);
```

```
40
41          // Max and Min
42          int max = numbers[0], min = numbers[0];
43          for (int i = 1; i < numbers.length; i++) {
44              if (numbers[i] > max) max = numbers[i];
45              if (numbers[i] < min) min = numbers[i];
46          }
47
48          System.out.println("Max = " + max);
49          System.out.println("Min = " + min);
50
51          // Searching
52          int search = sc.nextInt();
53          boolean found = false;
54
55          for (int v : numbers) {
56              if (v == search) {
57                  found = true;
58                  break;
59              }
60          }
61
62          System.out.println(found ? "Found" : "Not Found");
63
64          sc.close();
65      }
66  }
```

Lesson Program: Types of Arrays in Java.

Source Code:

TypesOfArrays.java

```java
// Program: More Array Topics in Java

public class TypesOfArrays {
    public static void main(String[] args) {

        // -------- Array of Objects --------
        Student[] students = new Student[3];

        students[0] = new Student("Amit", 85);
        students[1] = new Student("Neha", 90);
        students[2] = new Student("Rahul", 78);

        for (int i = 0; i < students.length; i++) {
            System.out.println(students[i].name + " - " + students[i].marks);
        }

        // -------- 2D Array --------
        int[][] numbers = {
                {10, 20, 30},
                {40, 50, 60}
        };

        for (int i = 0; i < numbers.length; i++) {
            for (int j = 0; j < numbers[i].length; j++) {
                System.out.print(numbers[i][j] + " ");
            }
            System.out.println();
        }
    }
}

// Student class
class Student {  5 usages
    String name;  2 usages
    int marks;  2 usages

    Student(String name, int marks) {  3 usages
        this.name = name;
        this.marks = marks;
    }
}
```

Lesson Program: Advance Topics in Arrays in Java.

Source Code:

AdvanceTopicsInArrays.java

```java
// Program: Advanced Topics in Arrays (Introduction)

public class AdvanceTopicsInArrays {
    public static void main(String[] args) {

        // ----- Sample Array -----
        int[] numbers = {5, 10, 15, 20};

        // ----- Simple Algorithm: Find Maximum -----
        int max = numbers[0];
        for (int n : numbers) {
            if (n > max) max = n;
        }
        System.out.println("Maximum value: " + max);

        // ----- Fixed Size -----
        System.out.println("Array size: " + numbers.length);

        // ----- Reference Behavior -----
        int[] ref = numbers;
        ref[0] = 99;
        System.out.println("numbers[0] after reference change: " + numbers[0]);

        // ----- Array Copying -----
        int[] copy = numbers.clone();
        copy[1] = 77;
        System.out.println("Original numbers[1]: " + numbers[1]);
        System.out.println("Copied copy[1]: " + copy[1]);

        // ----- Passing Array to Method -----
        printFirstElement(numbers);
    }

    static void printFirstElement(int[] arr) { 1 usage
        System.out.println("First element via method: " + arr[0]);
    }
}
```

// Java Arrays Summary

- An **array** stores **multiple values of the same data type** in a single variable.
- Elements are stored in **contiguous memory**.
- **Index starts from 0** and ends at size - 1.
- **Array size is fixed** once created.
- Arrays require:
  - Declaration
  - Creation
  - Initialization
- Arrays can be traversed using:
  - for
  - while
  - **enhanced for loop**
- Common operations:
  - Traversing
  - Updating
  - Sum & average
  - Max & min
  - Searching
- Arrays can store:
  - **Primitive values**
  - **Objects**
  - **2D data (rows & columns)**
- Arrays use **reference behavior** and are **mutable**.
- Fixed-size limitation leads to **ArrayList and Collections**.

Arrays are the foundation for Collections and Data Structures in Java.

Strings in Java

In programming, we frequently need to store, display, and manipulate **textual data** such as names, messages, passwords, addresses, and user input. Declaring separate variables for each character is impractical and inefficient. To handle such text-based data effectively, Java provides a powerful built-in class called **String**.

A **String** in Java is a sequence of characters stored as a **single object**. Unlike primitive data types, strings belong to the String class and are stored in **heap memory**. Java treats strings as special objects and provides extensive support for string handling, making text processing simple, safe, and efficient.

Strings in Java are **immutable**, which means once a string object is created, its value **cannot be changed**. Any modification to a string actually results in the creation of a **new string object**. This immutability improves **security, memory efficiency, and thread safety**, especially when strings are shared across different parts of a program.

Java optimizes string storage using a mechanism called the **String Constant Pool**. When string literals are created, Java checks whether the same value already exists in the pool. If it does, the existing object is reused. This reduces memory usage and improves performance, making string handling highly efficient.

Each character in a string is stored in a **sequential order** and can be accessed using an **index**, starting from 0. This allows direct access to individual characters, similar to arrays. However, unlike character arrays, strings provide many built-in methods for operations such as comparison, searching, formatting, and transformation.

Java supports different ways of working with strings, including:

- String literals
- Creating strings using the new keyword
- Mutable string classes like StringBuilder and StringBuffer

Strings play a **fundamental role** in Java programming and are widely used in:

- user input and output
- form validation
- file handling
- data processing
- networking and web applications
- logging and reporting systems

Understanding strings is essential before moving to advanced topics such as **string methods, string comparison, mutability vs immutability, collections, and data structures**. A strong foundation in strings helps in writing efficient, secure, and readable Java programs.

This chapter will explore strings step by step, starting from basic concepts and gradually moving toward advanced string handling techniques in Java.

Lesson Program: Strings in Java.

Source Code:

StringsInJava.java

```java
// Program: Strings in Java

public class StringsInJava {
    public static void main(String[] args) {

        // String declaration
        String s1;
        String s2;

        // String initialization using literal
        s1 = "Java";
        s2 = "Java";

        // Accessing string value
        System.out.println(s1);
        System.out.println(s2);

        // Reference comparison (memory)
        System.out.println(s1 == s2);   // true (same SCP object)

        // Creating string using new keyword
        String s3 = new String("Java");
        String s4 = new String("Java");

        // Accessing strings
        System.out.println(s3);
        System.out.println(s4);

        // Reference comparison
        System.out.println(s3 == s4);   // false (different objects)

        // String length
        System.out.println(s1.length());

        // Character access
        System.out.println(s1.charAt(0));

        // String concatenation
        String s5 = s1 + " Programming";
        System.out.println(s5);
    }
}
```

Lesson Program: String Methods in Java.

Source Code:

StringMethods.java

```java
// Program: String Methods in Java

public class StringMethods {
    public static void main(String[] args) {

        String text = "Java Programming";

        // length()
        System.out.println(text.length());

        // charAt()
        System.out.println(text.charAt(0));

        // substring()
        System.out.println(text.substring(5));
        System.out.println(text.substring(0, 4));

        // equals() vs ==
        String a = "Java";
        String b = new String("Java");
        System.out.println(a.equals(b)); // true
        System.out.println(a == b);      // false

        // case conversion
        System.out.println(text.toUpperCase());
        System.out.println(text.toLowerCase());

        // trim()
        String spaced = "  Hello Java  ";
        System.out.println(spaced.trim());

        // contains()
        System.out.println(text.contains("Java"));

        // startsWith() & endsWith()
        System.out.println(text.startsWith("Java"));
        System.out.println(text.endsWith("ming"));

```

```
39          // replace()
40          System.out.println(text.replace("Java", "Python"));
41
42          // split()
43          String[] words = text.split(" ");
44          System.out.println(words[0]);
45
46          // isEmpty()
47          String empty = "";
48          System.out.println(empty.isEmpty());
49      }
50  }
```

Lesson Program: String Mutability in Java.

Source Code:

MutableImmutableStrings.java

```java
// Program: Mutable & Immutable Strings in Java

public class MutableImmutableStrings {
    public static void main(String[] args) {

        // Immutable String
        String s = "Java";
        s.concat(" Programming");
        System.out.println(s); // Java

        // New object created
        s = s.concat(" Programming");
        System.out.println(s); // Java Programming

        // StringBuilder (Mutable)
        StringBuilder sb = new StringBuilder("Java");
        sb.append(" Programming");
        System.out.println(sb);

        // StringBuffer (Mutable)
        StringBuffer sbf = new StringBuffer("Java");
        sbf.append(" Programming");
        System.out.println(sbf);

        // String comparison
        String a = "Java";
        String b = new String("Java");

        System.out.println(a == b);        // false
        System.out.println(a.equals(b));   // true
    }
}
```

Lesson Program: String Builder and Buffer in Java.

Source Code:

StringBuilderAndBuffer.java

```java
// Program: StringBuilder & StringBuffer in Java

public class StringBuilderAndBuffer {
    public static void main(String[] args) {

        // StringBuilder (Mutable & Fast)
        StringBuilder sb = new StringBuilder("Java");
        sb.append(" Programming");
        System.out.println(sb);

        sb.insert(5, "Language ");
        System.out.println(sb);

        sb.replace(0, 4, "JAVA");
        System.out.println(sb);

        sb.delete(5, 14);
        System.out.println(sb);

        // StringBuffer (Mutable & Thread-safe)
        StringBuffer sbf = new StringBuffer("Java");
        sbf.append(" Programming");
        System.out.println(sbf);

        // Capacity & Length
        StringBuilder cap = new StringBuilder("Hello");
        System.out.println(cap.capacity());
        System.out.println(cap.length());
    }
}
```

// Java Strings Summary

- Strings are used to store and manipulate **textual data** such as names, messages, passwords, and user input.
- In Java, String is a **predefined class**, not a primitive data type, and it belongs to the java.lang package.
- Strings are **immutable**, which means once a String object is created, its value cannot be changed. Any modification results in the creation of a **new String object**.
- Strings can be created in two ways:
    - **String literals**, which are stored in the **String Constant Pool** and allow memory sharing.
    - **Using the new keyword**, which creates a new object in heap memory every time.
- The == operator compares **memory references**, not the actual content of strings.
- To compare string content, the **equals()** and **equalsIgnoreCase()** methods must be used.
- Java provides a rich set of **String methods** to inspect and manipulate text, such as:
    - length(), charAt(), substring()
    - toUpperCase(), toLowerCase(), trim(), split(), replace()
- Because strings are immutable, using String for frequent modifications can lead to **performance issues**.
- To handle mutable string operations efficiently, Java provides:
    - **StringBuilder**, which is fast but not thread-safe.
    - **StringBuffer**, which is thread-safe but slightly slower.
- Understanding string behaviour, memory usage, immutability, and comparison methods is **critical for writing correct, efficient, and secure Java programs**.

Strings form the backbone of Java applications and are used extensively in real-world development.

Methods in Java

In programming, many tasks need to be performed **repeatedly**, such as calculating results, validating input, printing formatted output, or processing data. Writing the same code again and again makes programs **lengthy, hard to read, difficult to maintain, and error-prone**. To solve this problem, Java provides a powerful concept called **methods**.

A **method** in Java is a **block of code that performs a specific task** and is executed only when it is called. Instead of rewriting the same logic multiple times, a method allows programmers to **write the logic once and reuse it wherever needed**. This promotes **code reusability, modularity, and clarity**, which are essential for writing clean and efficient programs.

Methods help break a large program into **smaller, manageable units**, where each method focuses on a single responsibility. This makes programs easier to **understand, test, debug, and modify**. For example, instead of writing login validation logic inside the main program repeatedly, it can be placed inside a method and reused whenever required.

In Java, methods belong to **classes** and are executed by calling them using their method name. A method can:

- take **input** using parameters
- perform **processing**
- optionally return a **result**

Some methods return values, while others simply perform actions such as printing output. Java also provides many **built-in methods** (like print(), length(), charAt()), and programmers can define their **own custom methods** to solve problems.

Methods in Java support important programming concepts such as:

- **Abstraction** (hiding implementation details)
- **Reusability** (use once, call many times)
- **Maintainability** (easy updates and fixes)
- **Structured programming** (clear program flow)

Understanding methods is **fundamental** before learning advanced topics such as **method overloading, recursion, object-oriented programming, and data structures**. Almost every real-world Java application relies heavily on methods to organize logic and manage complexity.

This chapter will explore **methods in Java step by step**, starting with basic method concepts and gradually moving toward more advanced method usage and applications.

Lesson Program: Methods in Java.

Source Code:

MethodsInJava.java

```java
// Program: Introduction to Methods in Java

public class MethodsInJava {

    // ---------------- Method with No Parameters ----------------
    static void greet() {  1 usage
        System.out.println("Hello! Welcome to Java Methods.");
    }

    // ---------------- Method with Parameters ----------------
    static void add(int a, int b) {  2 usages
        int sum = a + b;
        System.out.println("Sum = " + sum);
    }

    // ---------------- Method with Return Value ----------------
    static int square(int num) {  1 usage
        return num * num;
    }

    // ---------------- Method Returning Boolean ----------------
    static boolean isEven(int n) {  1 usage
        return n % 2 == 0;
    }

    // ---------------- Method Using String ----------------
    static void printMessage(String name) {  1 usage
        System.out.println("Hello, " + name);
    }

    // ---------------- Method with Multiple Operations ----------------
    static int max(int a, int b) {  1 usage
        if (a > b) {
            return a;
        }
        return b;
    }

```

```
    public static void main(String[] args) {

        // Calling method with no parameters
        greet();

        // Calling method with parameters
        add(10, 20);
        add(5, 7);

        // Calling method with return value
        int sq = square(4);
        System.out.println("Square = " + sq);

        // Using boolean return value
        boolean result = isEven(10);
        System.out.println("Is 10 even? " + result);

        // Passing String as argument
        printMessage("Aditya");

        // Method returning max value
        int bigger = max(15, 9);
        System.out.println("Maximum value = " + bigger);
    }
}
```

Lesson Program: Types of Methods in Java.

Source Code:

TypesOfMethods.java

```java
// Program: Types of Methods in Java

public class TypesOfMethods {

    // 1. No parameters, no return value
    static void greet() {  1 usage
        System.out.println("Welcome to Java!");
    }

    // 2. Parameters, no return value
    static void add(int a, int b) {  1 usage
        System.out.println("Sum = " + (a + b));
    }

    // 3. No parameters, return value
    static int getNumber() {  1 usage
        return 10;
    }

    // 4. Parameters and return value
    static int square(int n) {  1 usage
        return n * n;
    }

    // 5. Boolean return type
    static boolean isEven(int n) {  1 usage
        return n % 2 == 0;
    }

    public static void main(String[] args) {

        greet();                        // Type 1

        add(5, 7);                      // Type 2

        int num = getNumber();          // Type 3
        System.out.println("Number = " + num);

        int result = square(4);         // Type 4
        System.out.println("Square = " + result);

        System.out.println("Is 6 even? " + isEven(6)); // Type 5
    }
}
```

Lesson Program: Method Overloading in Java.

Source Code:

MethodOverloadingInJava.java

```java
// Program: Method Overloading in Java

public class MethodOverloading {

    // Same method name, different parameters

    static void show() {  1 usage
        System.out.println("show() with no parameters");
    }

    static void show(int a) {  1 usage
        System.out.println("show(int): " + a);
    }

    static void show(int a, int b) {  1 usage
        System.out.println("show(int, int): " + (a + b));
    }

    static void show(double x) {  1 usage
        System.out.println("show(double): " + x);
    }

    static int calculate(int a, int b) {  1 usage
        return a + b;
    }

    static double calculate(double a, double b) {  1 usage
        return a * b;
    }

    public static void main(String[] args) {

        show();
        show(10);
        show(5, 7);
        show(3.5);

        System.out.println("calculate(2, 3): " + calculate(2, 3));
        System.out.println("calculate(2.5, 4.0): " + calculate(2.5, 4.0));
    }
}
```

Lesson Program: Scope of Variables in Java.

Source Code:

ScopeOfVariablesInJava.java

```java
// Program: Scope of Variables in Java

public class ScopeOfVariables {

    // Instance variable
    int instanceVar = 100;  2 usages

    // Static variable
    static int staticVar = 200;  2 usages

    // Local variable example
    void showLocalScope() {  1 usage
        int localVar = 50;
        System.out.println("Local: " + localVar);
        System.out.println("Instance: " + instanceVar);
        System.out.println("Static: " + staticVar);
    }

    // Block scope example
    void blockScope() {  1 usage
        if (true) {
            int blockVar = 25;
            System.out.println("Block variable: " + blockVar);
        }
        // blockVar not accessible here
    }

    // Parameter scope example
    void parameterScope(int value) {  1 usage
        System.out.println("Parameter value: " + value);
    }

    public static void main(String[] args) {

        ScopeOfVariables obj1 = new ScopeOfVariables();
        obj1.showLocalScope();
        obj1.blockScope();
        obj1.parameterScope(99);

        ScopeOfVariables obj2 = new ScopeOfVariables();

        System.out.println("obj2 instanceVar: " + obj2.instanceVar);
        System.out.println("Shared staticVar: " + staticVar);
    }
}
```

Lesson Program: Arrays to Methods in Java.

Source Code:

ArraysToMethods.java

```java
// Program: Passing & Returning Arrays to/from Methods in Java

public class ArraysToMethods {

    // Print array
    static void printArray(int[] arr) {  3 usages
        for (int v : arr) {
            System.out.print(v + " ");
        }
        System.out.println();
    }

    // Modify array (in-place)
    static void doubleArray(int[] arr) {  1 usage
        for (int i = 0; i < arr.length; i++) {
            arr[i] *= 2;
        }
    }

    // Return a new array
    static int[] squareArray(int[] arr) {  1 usage
        int[] result = new int[arr.length];
        for (int i = 0; i < arr.length; i++) {
            result[i] = arr[i] * arr[i];
        }
        return result;
    }

    public static void main(String[] args) {

        int[] numbers = {2, 4, 6};

        // Passing array to method
        printArray(numbers);

        // Modifying array inside method
        doubleArray(numbers);
        printArray(numbers);

        // Returning array from method
        int[] squared = squareArray(numbers);
        printArray(squared);
    }
}
```

// Java Methods Summary

- A **method** is a named block of code that performs a specific task and can be executed whenever required.
- Methods help in **organizing code**, **reducing repetition**, and **improving readability** of programs.
- Java programs become **modular** when logic is divided into smaller methods instead of writing everything inside main().
- Methods can be classified based on:
    - Presence of parameters
    - Return type
    - Type of data they process
- **Parameters** act as placeholders in method definitions, while **arguments** are the actual values passed during method calls.
- Methods may **return a value** using the return keyword, or may return nothing using the void return type.
- Java supports **method overloading**, where multiple methods share the same name but differ in parameter list.
- Method overloading improves **code clarity** and provides **compile-time polymorphism**.
- Variables used inside methods follow **scope rules**:
    - Local variables exist only inside methods
    - Instance variables belong to objects
    - Static variables are shared across the class
- Arrays can be **passed to methods**, **modified inside methods**, and **returned from methods**, making methods powerful for data processing.
- Understanding methods is essential because:
    - Every Java program revolves around methods
    - main() itself is a method
    - Advanced concepts like OOP, recursion, and DSA depend on method usage

Methods are the foundation of structured and reusable programming in Java, enabling clean logic, modular design, and efficient code management.